

Machine Learning Engineer Candidate Project

Introduction

We want the interview process to – as best as possible – reflect the realities of working in the real world. In a typical real-world scenario, you would be given project-based work as part of a team and will have time to perform research to solve the assigned task. As such, our interviews are based on projects which emulate this workflow.

We also believe that success in a project includes communication of the solution to stakeholders. Once you've completed the project, you will present your solution to our team. The context of this project mocks a real-world problem involving real estate. When you present to us, we would like you to pretend that we are real estate professionals rather than engineers. Once you've finished that presentation, we will dig deeper into the technical details of the solution.

In other words, your presentation should be split into two parts:

- One part where you will prepare a 10 minute presentation covering the high-level goals of the application, aimed at a non-technical audience, NOT engineers or data scientists. Do NOT fixate on the specific changes you made like data imputation, instead you should focus on the business goals of the application. Pretend as though the audience is a business stakeholder who is brand new to the project.
- A second part detailing technical aspects of the solution, aimed at engineers. We expect a live demo + walkthrough, but NO prepared slides. Be ready for an interactive discussion about your implementation, trade-offs, and LLM usage.
- NOTE: If you are applying to an architect level position you should include supplementary documentation like an architecture diagram and be prepared to discuss how you would deploy this API in a production environment.

We will schedule one hour for this presentation, but you need not present for a full hour – we generally ask a lot of questions.

We would like to have the opportunity to review your solution before presenting. Please add your project to a private GitHub repository and share the link with our recruiter. If you don't have a GitHub account, you can create one for free at github.com.

Project Scenario

Sound Realty helps people sell homes in the Seattle area. They have been using a basic machine learning model to estimate property values, deployed as a REST API. While the initial deployment has been successful, they've identified several areas for improvement:

1. **Missing Data Handling:** Real-world property data often has missing values. The current system cannot handle incomplete data, which limits its usefulness.
2. **Performance Issues:** The API is slower than expected, especially under load. Response times need to be improved for a better user experience.

3. **Code Quality:** As the system has evolved, some technical debt has accumulated that should be addressed.

Sound Realty has contracted us to enhance the existing system. Our data science team has already conducted research on imputation methods (see `notebooks/imputation_experiment.ipynb`) and identified KNN imputation as the preferred approach. Your job is to implement these improvements and prepare the system for production use.

Current System

The existing system includes:

- **FastAPI REST Service:** Deployed via Docker, serving predictions at the `/predict` endpoint
- **Machine Learning Model:** A trained model (`src/model/model.pkl`) with associated feature metadata
- **Data Files:**
 - `src/data/kc_house_data.csv` – Historical home sales data
 - `src/data/zipcode_demographics.csv` – Demographic data joined by zipcode
 - `src/data/future_unseen_examples.csv` – Test examples for validation
- **Jupyter Notebook:** Research on imputation methods (`notebooks/imputation_experiment.ipynb`)

Current API Behavior

The `/predict` endpoint accepts JSON POST requests with home features:

```
{
  "bedrooms": 3,
  "bathrooms": 2.5,
  "sqft_living": 2000,
  "sqft_lot": 5000,
  "floors": 2,
  "sqft_above": 1500,
  "sqft_basement": 500,
  "zipcode": "98125"
}
```

The API returns a prediction:

```
{
  "predicted_price": 450000.0
}
```

Deliverables/Requirements

1. Implement Missing Data Handling

Implement the imputation logic researched in the Jupyter notebook ([notebooks/imputation_experiment.ipynb](#)). The system should:

- Accept requests with missing values for any field except **zipcode** (which is always required)
- Use KNN imputation to fill in missing values before making predictions
- Handle missing data gracefully without errors
- Document the imputation approach in your presentation

Example request with missing data:

```
{
  "bedrooms": 3,
  "bathrooms": null,
  "sqft_living": 2000,
  "sqft_lot": null,
  "floors": 2,
  "sqft_above": 1500,
  "sqft_basement": 500,
  "zipcode": "98125"
}
```

2. Improve API Performance

The current implementation has performance issues. Identify and fix the bottlenecks to improve response times. Consider:

- How resources are loaded and managed
- Opportunities for caching or preloading
- Efficient data handling practices

3. Code Quality Improvements

Review the codebase and implement improvements that would make it more production-ready. This might include:

- Better error handling
- Input validation
- Logging and monitoring capabilities
- Code organization and structure
- Documentation
- Testing improvements
- Dependency management

4. Testing and Validation

- Update or create tests that validate your changes
- Demonstrate that the imputation logic works correctly
- Show that performance has improved
- Validate that the API still produces accurate predictions

5. Documentation

Update the README and other documentation to reflect your changes. Include:

- How to run the updated system
- How the imputation logic works
- What performance improvements were made
- Any new dependencies or requirements

Recommendations

- **Start Simple:** Get the basic imputation working first, then optimize
- **Measure Performance:** Use timing/profiling to identify actual bottlenecks before optimizing
- **Use Existing Tools:** Leverage scikit-learn's `KNNImputer` as demonstrated in the notebook
- **Docker First:** Ensure your changes work in the Docker environment
- **Test Thoroughly:** Use the examples in `future_unseen_examples.csv` for testing
- **Use AI Tools:** Feel free to use AI coding assistants (GitHub Copilot, ChatGPT, Claude, etc.) to help with implementation. We use these tools in our daily work and want to see how you leverage them effectively. Be prepared to discuss how you used AI tools and validate that you understand the code you're submitting.

Non-Requirements

- **Completing in a specific amount of time:** Life is busy and chaotic. We understand you will not be able to work full time on this project.
- **Retraining the model:** The existing model is fine. Focus on the deployment and data handling aspects.
- **Cloud deployment:** Everything can be done on a laptop running Docker Desktop, there is no need to deploy your code to a cloud service.
- **Perfect optimization:** We're looking for meaningful improvements, not perfection. Focus on the most impactful changes.
- **An exact end result:** Two candidates given this assignment will find different solutions. Feel free to choose your own adventure as long as the base requirements are met.

Time Management

We cannot stress these two enough:

1. **Build the simplest possible solution first**, utilizing tools you are familiar with when possible.
2. **Don't get stuck on one aspect of the project.** Ask questions and use the internet for research. Focus on your core strengths.

Evaluation Criteria

We will evaluate your submission based on:

- **Functionality:** Does the imputation logic work correctly? Are performance improvements measurable?
- **Code Quality:** Is the code well-organized, readable, and maintainable?

- **Problem Solving:** How did you approach the challenges? What trade-offs did you consider?
- **Communication:** Can you explain your solution to both technical and non-technical audiences?
- **Production Readiness:** How close is this to something you'd deploy in production?

AI Usage Expectations

We expect that you will use some form of AI coding assistant or agent to solve this problem, as it is now a reality of software engineering work. As you discuss your solution with our team, you should be prepared to:

1. Describe how you used AI to solve the problem
2. Share any challenges you faced and techniques you used in managing context and generated code
3. Nerd out on how your favorite AI tools work best for you

Getting Started

1. Clone the repository and explore the existing code
2. Review the Jupyter notebook to understand the imputation research
3. Run the existing Docker container to understand current behavior
4. Identify performance bottlenecks (hint: look at what happens on each request)
5. Implement imputation logic
6. Test your changes thoroughly
7. Document your work

One More Thing

We wish you all the best as you work on this project and thank you again for your interest in our team. If you have any suggestions for this project or our interview process, please give us feedback. Our goal is to make the interview process a positive experience for candidates and we are always interested in improving.

Good luck!